

OBJECT-ORIENTED PROGRAMMING (OOP) COURSEWORK REPORT

Topic: Employee Payroll and Bonus Management System

Source Code Repository (GitHub): [*OOP Labw 04*](#)

PART A: ANALYSIS AND DESIGN REPORT

A.1 Class Design

- System classes:
 - `Employee` : Base employee class
 - `SalariedEmployee` : Fixed-salary employee
 - `HourlyEmployee` : Hourly-waged employee
 - `SalesEmployee` : Commissioned sales employee
 - `Payroll` : Payroll management
- Class relationships:
 - **Inheritance:**
`SalariedEmployee` , `HourlyEmployee` , and `SalesEmployee` inherit from the abstract base class `Employee` .
 - **Aggregation:**
`Payroll` maintains a reference list `List<Employee>` . `Payroll` is not a subtype of `Employee` and maintains a lifecycle independent of individual `Employee` instances.

1. The `Employee` Class

- **Responsibilities:** Manages identity details, department assignments, and accumulated bonuses; provides a polymorphic interface for gross pay calculations.
- **Attributes:**
 - `EmployeeId`: `string` (`public get` , `private set`) - Unique employee identifier.
 - `FullName`: `string` (`public get` , `public set`) - Full name.
 - `Department`: `string` (`public get` , `public set`) - Department.
 - `MonthlyBonus`: `double` (`public get` , `private set`) - Total cumulative bonus for the month.

- **Business Invariants:**

- `EmployeeId`, `FullName`, and `Department` must neither be empty nor consist solely of whitespace.
- `MonthlyBonus >= 0`.

- **Constructors:**

- `Employee(string employeeId, string fullName) : Defaults Department = "Unassigned", MonthlyBonus = 0`. Delegates to the canonical constructor.
- `Employee(string employeeId, string fullName, string department) : Initialises valid fields, with MonthlyBonus = 0`.

- **Methods:**

- `AddBonus(double amount) : void : Fixed bonus increment ( amount > 0 )`.
- `AddBonus(double amount, string reason) : void :`
Fixed bonus increment with specified justification (amount > 0 ; reason non-empty).
- `AddBonus(double rate, double referenceAmount, string reason) : void :`
Percentage-based bonus (0 < rate <= 0.5 ; referenceAmount > 0 ; reason non-empty).
- `ResetBonus() : void : Resets MonthlyBonus = 0` at the start of a new payroll cycle.
- `abstract CalculateGrossPay() : double : Abstract method computing pre-tax gross earnings`.
- `abstract GetEmployeeType() : string : Abstract method returning the employee category descriptor`.
- `virtual DisplayPayrollInfo() : void : Outputs base payroll details for the employee`.

- **Bonus Design Strategy:** The system solely accumulates values into `MonthlyBonus` without persisting transaction histories, as the required domain scope is confined to monthly gross pay aggregation.

2. The `SalariedEmployee` Class

- **Responsibilities:** Manages remuneration for employees receiving a fixed monthly salary and responsibility allowance.
- **Inheritance:** `Employee`.
- **Additional Attributes:**
 - `MonthlySalary : double ( public get , private set )` - Fixed monthly wage.
 - `ResponsibilityAllowance : double ( public get , private set )` - Responsibility allowance.

- **Business Invariants:**

- Inherits all invariants from `Employee`.
- `MonthlySalary >= 0`.
- `ResponsibilityAllowance >= 0`.

- **Constructors:**

- `SalariedEmployee(string employeeId, string fullName, double monthlySalary)`: Defaults `ResponsibilityAllowance = 0`, `Department = "Unassigned"`.
- `SalariedEmployee(string employeeId, string fullName, string department, double monthlySalary, double responsibilityAllowance)`: Full attribute initialiser.

- **Overridden Methods:**

- `override CalculateGrossPay(): double`:
Gross pay calculation: `MonthlySalary + ResponsibilityAllowance + MonthlyBonus`.
- `override GetEmployeeType(): string`: Returns `"Salaried"`.
- `override DisplayPayrollInfo(): void`: Displays a breakdown including salary, allowance, bonus, and total gross pay.

3. The `HourlyEmployee` Class

- **Responsibilities:** Manages remuneration calculated from an hourly rate and overtime hours exceeding standard thresholds.

- **Inheritance:** `Employee`.

- **Additional Attributes:**

- `HourlyRate: double` (`public get`, `private set`) - Standard hourly pay rate.
- `WorkedHours: double` (`public get`, `private set`) - Total hours logged in the month.

- **Business Invariants:**

- Inherits all invariants from `Employee`.
- `HourlyRate >= 0`.
- `0 <= WorkedHours <= 250`.
- Does not store a dedicated field for overtime pay (derived dynamically from object state).

- **Constructors:**

- `HourlyEmployee(string employeeId, string fullName, double hourlyRate)`: Defaults `WorkedHours = 0`, `Department = "Unassigned"`.
- `HourlyEmployee(string employeeId, string fullName, string department, double hourlyRate, double workedHours)`: Full attribute initialiser.

- **Overridden Methods:**

- `override CalculateGrossPay(): double :`
 - If `WorkedHours <= 160` : `BasePay = WorkedHours * HourlyRate` .
 - If `WorkedHours > 160` :
`BasePay = (160 * HourlyRate) + ((WorkedHours - 160) * HourlyRate * 1.5)` .
 - Gross earnings: `BasePay + MonthlyBonus` .
- `override GetEmployeeType(): string : Returns "Hourly"` .
- `override DisplayPayrollInfo(): void :` Displays hourly rate, regular hours, overtime hours (if applicable), bonus, and total gross pay.

4. The `SalesEmployee` Class

- **Responsibilities:** Manages remuneration comprising a base salary and sales commission.
- **Inheritance:** `Employee` .
- **Additional Attributes:**
 - `BaseSalary: double ( public get , private set )` - Base salary.
 - `SalesRevenue: double ( public get , private set )` - Sales turnover generated.
 - `CommissionRate: double ( public get , private set )` - Commission percentage.
- **Business Invariants:**
 - Inherits all invariants from `Employee` .
 - `BaseSalary >= 0` .
 - `SalesRevenue >= 0` .
 - `0 <= CommissionRate <= 0.3` .
- **Constructors:**
 - `SalesEmployee(string employeeId, string fullName, double baseSalary, double commissionRate) :` Defaults `SalesRevenue = 0` , `Department = "Unassigned"` .
 - `SalesEmployee(string employeeId, string fullName, string department, double baseSalary, double salesRevenue, double commissionRate) :` Full attribute initialiser.
- **Additional & Overridden Methods:**
 - `UpdateSalesRevenue(double revenue): void :` Updates total revenue (`revenue >= 0`).
 - `override CalculateGrossPay(): double :`
Gross pay calculation: `BaseSalary + (SalesRevenue * CommissionRate) + MonthlyBonus` .

- `override GetEmployeeType(): string`: Returns "Sales" .
- `override DisplayPayrollInfo(): void`: Displays base salary, revenue, commission rate, accrued commission, bonus, and total gross pay.

5. The Payroll Class

- **Responsibilities:** Coordinates the registry of employees for a designated payroll period; conducts summary aggregations and payroll reporting.
- **Attributes:**
 - `Period: string` (`public get` , `private set`) - Target payroll period (e.g. 2026-09).
 - `Employees: List<Employee>` (`public get` , `private set`) - Collection of references to employee objects.
- **Business Invariants:**
 - `Period` must neither be empty nor consist solely of whitespace.
 - No duplicate `EmployeeId` instances may exist within the `Employees` collection.
- **Constructors:**
 - `Payroll(string period)` : Initialises the period identifier alongside an empty employee collection.
- **Methods:**
 - `AddEmployee(Employee employee): bool` : Enrolls an employee into the payroll (returns `false` if `EmployeeId` already exists; otherwise adds and returns `true`).
 - `FindEmployee(string employeeId): Employee?` : Locates an employee by identifier (returns `null` if not found).
 - `CalculateTotalPayroll(): double` : Calculates total expenditure across the payroll via polymorphic invocations of `CalculateGrossPay()` .
 - `CalculatePayrollByDepartment(string department): double` : Calculates cumulative gross pay for a specified department.
 - `FindHighestPaidEmployee(): Employee?` : Identifies the top-earning employee (returns `null` if the collection is empty).
 - `DisplayPayroll(): void` : Renders the complete payroll overview for the period.

A.2 Design Decisions

Design Questions:

1. Why is `calculateGrossPay()` suited for overriding rather than overloading?

- Gross pay calculation represents an identical conceptual responsibility whose execution logic differs across employee classifications; thus, inheriting the contract from the base class and overriding it across subclasses is required.
- Overloading would require an unmanageable proliferation of signatures (to accommodate different employee types with varying optional data profiles) and cannot resolve ambiguities when two method signatures share identical parameter types and counts, introducing significant maintenance overheads.

2. Why is `addBonus()` suited for overloading?

- In this design, `AddBonus` does not vary by subclass behaviour, but rather depends on distinct invocation parameter signatures available to any employee instance; hence, multiple overloaded versions are declared in the base class for direct consumption by all derived instances.

3. How should `Payroll` store objects, references, or pointers to avoid object slicing?

- It must not store values as concrete base objects, as this triggers Object Slicing (where derived instances are truncated to the memory footprint and layout of the base class, discarding derived state and virtual method dispatch); additionally, unnecessary value copying incurs runtime overhead.
- In `C++`, this is resolved by using pointers (or smart pointers) and reference semantics to avoid value copying and preserve polymorphism.
- In `C#`, classes are reference types by default, natively preserving object identity and dynamic dispatch via references.

4. Should the base class `Employee` be concrete or abstract? Justify the decision.

- It should be declared `abstract`. From a domain modelling standpoint, every employee must belong to a specific operational category; direct instantiation of a generic base employee would violate domain rules.

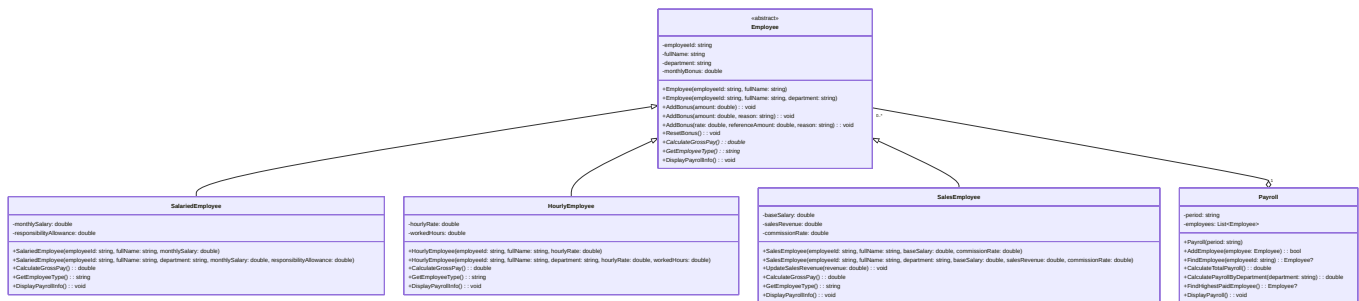
5. If sales employees were also remunerated hourly, would single inheritance remain appropriate?

- It would no longer be suitable, as single inheritance would inevitably induce duplicate code across disparate hierarchies.
- In `C#`, interfaces or compositional patterns can be leveraged, allowing a class to assemble multiple remuneration calculation strategies modularly.

6. Should `calculateTotalPayroll()` be declared as a static member? Why or why not?

- It should not. `CalculateTotalPayroll` operates strictly on a concrete context (a specific payroll period and its associated employee collection), meaning the computation is intrinsically coupled to individual object state.

A.3 UML Class Diagram



PART B: IMPLEMENTATION REPORT

The system has been implemented in `C#` adhering strictly to the design specifications and object-oriented principles detailed above.

Verification against coursework requirements:

B.1 The `Employee` Class

- **Attribute Encapsulation:** Implemented using `private` backing fields and property accessors (`get`, `private set`). Assigning empty or whitespace-only strings to `EmployeeId`, `FullName`, or `Department`, or supplying negative values to `MonthlyBonus`, is prevented by raising exceptions.
- **Constructor Overloading:** Provides two constructors. The concise constructor employs `: this(...)` syntax to delegate to the canonical constructor, eliminating duplicate validation logic.
- **Overloading `AddBonus()`:** Implements three overloaded signatures fulfilling the specification: `AddBonus(amount)`, `AddBonus(amount, reason)`, and `AddBonus(rate, referenceAmount, reason)`.
- **Bonus Reset:** Exposes a `ResetBonus()` method to reset the value back to `0.0` for subsequent payroll periods.
- **Polymorphic Declarations:** Declares `CalculateGrossPay()` and `GetEmployeeType()` as `abstract`, obligating derived classes to supply concrete overrides.

B.2 Derived Classes of `Employee`

For the classes: `SalariedEmployee`, `HourlyEmployee`, and `SalesEmployee`:

- **Constructors:** Each class implements two constructors, utilising `: base(...)` to pass shared identity fields up to the base class while evaluating subclass-specific invariants internally.
- **Data Validation:** Enforces invariants by rejecting negative monetary values (salary, allowances, hourly rate, revenue). Restricts `WorkedHours` to the range `[0, 250]` and `CommissionRate` to `[0, 0.3]`.
- **Method Overriding (`override`):** Implements `CalculateGrossPay()` in accordance with specified formulas. `HourlyEmployee` computes overtime pay directly from `WorkedHours` state without retaining redundant tracking fields.
- **Revenue Updates:** `SalesEmployee` provides an `UpdateSalesRevenue(double revenue)` method that overwrites the total and inherently triggers the non-negative guard inside the `SalesRevenue` property setter.

B.3 The `Payroll` Class

- **Collection Management:** Built upon `List<Employee>`.
- **Duplicate Prevention:** `AddEmployee()` checks the existing collection, returning `false` and rejecting additions upon encountering duplicate `EmployeeId` values.
- **Polymorphic Aggregation and Display:** `CalculateTotalPayroll()` and `DisplayPayroll()` iterate over records via `foreach` loops, invoking polymorphic pay calculations and display routines directly.
- **Departmental Aggregation:** Performs case-insensitive matching on department names. Returns `0` if a department does not exist or has accrued no gross pay.
If the input string is `null` or whitespace-only, an exception is raised.
- **Empty Collection Safeguards:** Both search (`FindHighestPaidEmployee`) and display (`DisplayPayroll`) guard against empty collections by evaluating `Employees.Count == 0` first, returning `null` or emitting clear messages without triggering a `NullReferenceException`.

PART C: TEST REPORT

C.1 Initialisation and Standard Business Logic Testing

The system initialises the payroll period 2026-09 and registers four employee instances representing distinct compensation models in accordance with the specification.

• **Employee Test Data Breakdown:**

ID	Full Name	Department	Employee Type	Remuneration Component Data	AddBonus() Overload Applied	1
E001	Name01	Training	Salaried	- Monthly salary: ... - Allowance: ...	AddBonus(double) (Overload 1: Fixed)	
E002	Name02	Support	Hourly	- Hourly rate: ... - Worked hours: 150h	AddBonus(double, string) (Overload 2: With reason)	
E003	Name03	Support	Hourly	- Hourly rate: ... - Worked hours: 170h	No bonus applied	
E004	Name04	Sales	Sales	- Base salary: ... - Revenue: ... - Commission: 5%	AddBonus(double, double, "...") (Overload 3: Rate by reference)	

• **Test Results:** (Expected to be successful)

C.2 Boundary and Exception Testing

The test suite in Test.cs validates invariant enforcement and verifies exception handling across boundary and error conditions:

No.	Test Scenario	Simulated Input Data	Expected Outcome
TC01	Instantiate employee with empty or whitespace ID	employeeId = " "	Throws ArgumentException
TC02	Apply negative bonus via	amount = -250000	Throws ArgumentOutOfRangeException

No.	Test Scenario	Simulated Input Data	Expected Outcome
	AddBonus(amount)		
TC03	Apply percentage bonus exceeding ceiling (> 0.5)	rate = 0.55 (55%)	Throws ArgumentOutOfRangeException
TC04	Instantiate salaried employee with negative monthly salary	monthlySalary = -5000000	Throws ArgumentOutOfRangeException
TC05	Hourly employee hours worked exceeding threshold (> 250)	workedHours = 250.5	Throws ArgumentOutOfRangeException
TC06	Commission rate exceeding threshold (> 0.3)	commissionRate = 0.35 (35%)	Throws ArgumentOutOfRangeException
TC07	Add duplicate employee identifier to payroll	Add E001 a second time	AddEmployee() returns false ; addition rejected
TC08	Add null object reference (null) to payroll	payroll.AddEmployee(null)	Throws ArgumentNullException
TC09	Calculate departmental payroll with empty/whitespace string	department = " "	Throws ArgumentException
TC10	Safe execution when payroll contains no employee records	Employees.Count == 0	Returns null for top earner; total payroll = 0

- **Test Results:**

```
>>> PART 2: INVARIANT, BOUNDARY & EXCEPTION TEST SUITE <<<
Verifying defensive programming contracts and exception handling...

[Test 1] TC01: Reject creation of Employee with blank or whitespace ID
-> SUCCESS: Caught expected ArgumentException: Employee ID must not be empty or contain only whitespace.

[Test 2] TC02: Reject negative bonus allocation in AddBonus(amount)
-> SUCCESS: Caught expected ArgumentOutOfRangeException [amount]: Bonus amount must be strictly greater than zero. (Parameter 'amount')

[Test 3] TC03: Reject bonus percentage exceeding 0.5 in AddBonus(rate, reference, reason)
-> SUCCESS: Caught expected ArgumentOutOfRangeException [rate]: Bonus rate must be greater than 0 and cannot exceed 0.5 (50%). (Parameter 'rate')

[Test 4] TC04: Reject negative monthly salary for SalariedEmployee
-> SUCCESS: Caught expected ArgumentOutOfRangeException [value]: Monthly salary cannot be negative. (Parameter 'value')

[Test 5] TC05: Reject worked hours exceeding monthly ceiling of 250 hours
-> SUCCESS: Caught expected ArgumentOutOfRangeException [value]: Worked hours must be between 0 and 250. (Parameter 'value')

[Test 6] TC06: Reject commission rate exceeding 0.3 (30%) for SalesEmployee
-> SUCCESS: Caught expected ArgumentOutOfRangeException [value]: Commission rate must be between 0 and 0.3 (30%). (Parameter 'value')

[Test 7] TC07: Reject duplicate Employee ID insertion into Payroll
-> SUCCESS: Correctly rejected duplicate Employee ID 'E001' (returned false).

[Test 8] TC08: Reject adding null Employee reference to Payroll
-> SUCCESS: Caught expected ArgumentNullException [employee]: Cannot add a null employee to the payroll. (Parameter 'employee')

[Test 9] TC09: Reject blank department query string in CalculatePayrollByDepartment
-> SUCCESS: Caught expected ArgumentException: Department name must not be empty or contain only whitespace.

[Test 10] TC10: Safe behaviour verification on completely empty Payroll instance
-> SUCCESS: Returned null for highest earner and 0 for grand total.

=====
ALL VERIFICATION PROCEDURES CONCLUDED
=====
```

bunny-os@computer:~/Satellites/Development_Lab/Workspaces/20261_study/OOP/OOP_Exercise_04\$

C.3 Test Evaluation and Analysis

1. Which `addBonus()` overload is resolved in each call, and why?

- Overload Resolution:

- `e001.AddBonus(double amount)` (Overload 1)
- `e002.AddBonus(double amount, string reason)` (Overload 2)
- `e004.AddBonus(double rate, double referenceAmount, string reason)` (Overload 3)

- Justification: Matches the argument count, types, and sequence of the formal parameter signatures.

2. Does method overload resolution occur at compile time or runtime?

- It takes place at compile time. The compiler examines the call site's arguments (count, types, and order) to bind the static method signature directly when emitting Common Intermediate Language (CIL).

- Mechanism: Static Binding / Early Binding

3. How is the derived class `calculateGrossPay()` method resolved?

- Resolution occurs at runtime. The compiler emits a virtual dispatch call (`callvirt`). During execution, the Common Language Runtime (CLR) inspects the virtual method table (`vtable`) of the concrete object allocated on the heap, dynamically dispatching to the overridden implementation corresponding to its runtime type.

- Mechanism: Dynamic Dispatch / Late Binding

Note: Method overriding differs fundamentally from overloading in its runtime dispatch timing.

4. If the single `Employee` list were replaced with separate lists for each subtype, how would the architecture be affected?

- *Loss of Polymorphism:* `Payroll` would no longer manage entities via the unified `Employee` abstraction.
- *Code Duplication:* Aggregation routines (`CalculateTotalPayroll` , `FindHighestPaidEmployee` , etc.) would have to be reimplemented redundantly for every distinct collection (`List<SalariedEmployee>` , `List<HourlyEmployee>` , `List<SalesEmployee>`).
- *Impaired Maintainability:* Adding any subsequent employee subtype would necessitate modifications across the `Payroll` class (introducing a new collection and amending all aggregation methods).

(From a domain modelling standpoint, employee variants are not disjoint entities; they share a common base contract: `Employee`)

5. When introducing a `ContractEmployee` class, which components must be modified and which should remain untouched?

- ***Components that should remain unmodified:***
 - The base class `Employee` .
 - Existing derived classes (`SalariedEmployee` , `HourlyEmployee` , `SalesEmployee`).
 - Aggregation and management logic inside `Payroll` (`CalculateTotalPayroll` , `CalculatePayrollByDepartment` , `FindHighestPaidEmployee` , etc.). Leveraging polymorphism, `Payroll` operates on `ContractEmployee` through the base reference without requiring modifications to calculation workflows.
- ***Components requiring modification or addition:***
 - Implement the new `ContractEmployee` class derived from `Employee` , overriding `CalculateGrossPay()` and `GetEmployeeType()` .
 - Extend test fixtures: Instantiate and incorporate `ContractEmployee` test cases into the verification suite.